FOOD SUPPLY CHAIN STOCK RECONCILIATION

Python Programming Project Report

Project topic	Food Supply Chain Stock Reconciliation
Language	Python 3
Purpose	Compare stock records, find discrepancies, and calculate replenishment needs across locations

Includes problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

1. Problem Statement

Develop a Python program to reconcile food stock records across multiple supply locations. For each location and food item, the program records opening stock, quantities received, distributed, damaged, and the recorded remaining quantity. It calculates the expected remaining stock, compares it with the recorded quantity, identifies shortages or surpluses, and calculates how much stock is needed to reach a specified minimum stock level. Finally, it presents an overall summary.

2. Objectives

• Collect stock movement details for multiple locations.

• Calculate expected remaining stock using opening stock + received – distributed – damaged.

• Compare expected stock with the recorded stock and identify shortages or surpluses.

• Calculate replenishment needed to reach the desired minimum stock level.

• Validate inputs and reject negative or non-numeric quantities.

• Summarize location-level results and overall discrepancy and replenishment totals.

3. Assumptions

• All quantities use the same unit for a given food item, such as kilograms, packets, or boxes.

• Quantities are whole numbers and cannot be negative.

• Each location is evaluated independently; the program does not automatically transfer stock between locations.

• Discrepancy is defined as recorded remaining stock minus expected remaining stock.

• A negative discrepancy indicates a shortage; a positive discrepancy indicates a surplus.

• Replenishment equals max(0, minimum desired stock – recorded remaining stock).

• This is an educational program and does not connect to a live inventory system.

4. Algorithm

1. Start and read the number of locations.

2. Validate that the location count is a positive integer.

3. For each location, read its name, food item, opening stock, received, distributed, damaged, recorded remaining, and minimum stock.

4. Validate that all quantity inputs are whole numbers and non-negative.

5. Calculate expected remaining = opening stock + received – distributed – damaged.

6. Calculate discrepancy = recorded remaining – expected remaining.

7. If discrepancy is zero, report reconciled; if negative, report shortage; if positive, report surplus.

8. Calculate replenishment = max(0, minimum stock – recorded remaining).

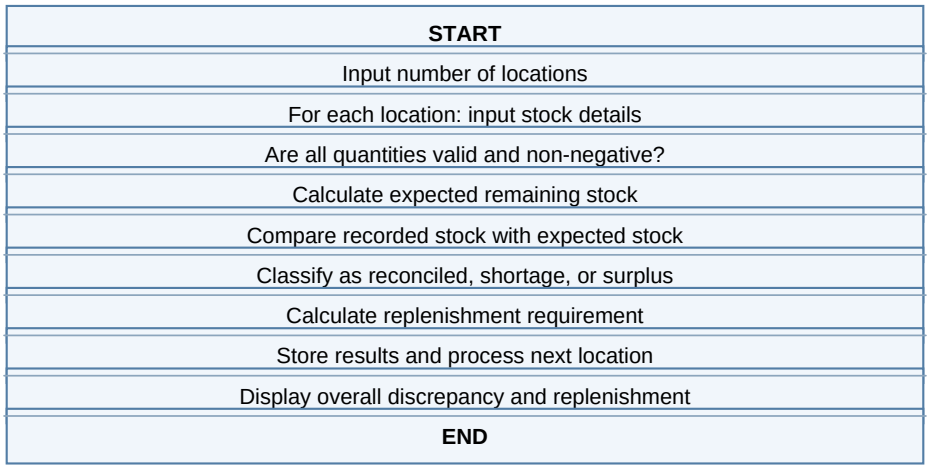
9. Store the valid location report and continue to the next location.

10. Display the number of successfully processed locations, net discrepancy, and total replenishment required.

11. Stop.

5. Flowchart

This flowchart summarizes the processing loop for each location and the final aggregation stage.



Flow: Start → Read locations → Enter and validate stock movements → Calculate expected stock → Compare and classify discrepancy → Calculate replenishment → Repeat → Overall summary → End.

6. Pseudocode

```
BEGIN
    INPUT location_count
    IF location_count is not a positive integer:
        DISPLAY error
        STOP

    reports = empty list

    FOR each location:
        INPUT location_name, food_item
        INPUT opening_stock, received, distributed, damaged
        INPUT recorded_remaining, minimum_stock

        IF any quantity is non-numeric or negative:
            DISPLAY input error
            SKIP this location
        ELSE:
            expected = opening_stock + received - distributed - damaged
            discrepancy = recorded_remaining - expected
            replenishment = MAX(0, minimum_stock - recorded_remaining)

            IF discrepancy = 0:
                status = "Reconciled"
            ELSE IF discrepancy < 0:
                status = "Shortage"
            ELSE:
                status = "Surplus"

            DISPLAY expected, recorded, discrepancy, status
            DISPLAY replenishment
            SAVE location report
        ENDIF
    END FOR

    DISPLAY successful locations
    DISPLAY sum of discrepancies
    DISPLAY sum of replenishment quantities
END
```

7. Python Source Code

Save the code as **food_stock_reconciliation.py** and run it with Python 3. For this simple version, if a location contains invalid quantity input, that location is skipped.

```
def reconcile_location():
    print("\\nEnter details for a food supply location.")
    location = input("Location name: ").strip()
    item = input("Food item: ").strip()

    try:
        opening_stock = int(input("Opening stock (units): "))
        received = int(input("Quantity received (units): "))
        distributed = int(input("Quantity distributed (units): "))
        damaged = int(input("Quantity damaged (units): "))
        recorded_remaining = int(input("Recorded remaining stock (units): "))
        minimum_stock = int(input("Minimum desired stock (units): "))
    except ValueError:
        print("Invalid input: quantities must be whole numbers.")
        return None

    quantities = [opening_stock, received, distributed, damaged,
                  recorded_remaining, minimum_stock]
    if any(q < 0 for q in quantities):
        print("Invalid input: quantities cannot be negative.")
        return None

    expected_remaining = opening_stock + received - distributed - damaged
    discrepancy = recorded_remaining - expected_remaining
    replenishment = max(0, minimum_stock - recorded_remaining)

    print("\\n--- Reconciliation Report ---")
    print(f"Location: {location}")
    print(f"Food item: {item}")
    print(f"Expected remaining stock: {expected_remaining}")
    print(f"Recorded remaining stock: {recorded_remaining}")
    print(f"Discrepancy (recorded - expected): {discrepancy}")

    if discrepancy == 0:
        print("Stock status: Reconciled")
    elif discrepancy < 0:
        print(f"Stock status: Shortage of {abs(discrepancy)} unit(s)")
    else:
        print(f"Stock status: Surplus of {discrepancy} unit(s)")

    print(f"Replenishment required to reach minimum stock: {replenishment}")
    return {
        "location": location,
        "item": item,
        "expected": expected_remaining,
        "recorded": recorded_remaining,
        "discrepancy": discrepancy,
        "replenishment": replenishment
    }


def main():
    print("=== Food Supply Chain Stock Reconciliation ===")
    try:
        count = int(input("Number of locations to process: "))
        if count <= 0:
            print("Enter at least one location.")
            return
    except ValueError:
        print("Invalid input: enter a whole number.")
        return

    reports = []
    for i in range(count):
        print(f"\\nLocation {i + 1}")
        report = reconcile_location()
        if report is not None:
            reports.append(report)

    print("\\n=== Overall Summary ===")
    print(f"Locations processed successfully: {len(reports)}")
    total_discrepancy = sum(r["discrepancy"] for r in reports)
    total_replenishment = sum(r["replenishment"] for r in reports)
    print(f"Net discrepancy across locations: {total_discrepancy} unit(s)")
    print(f"Total replenishment required: {total_replenishment} unit(s)")


if __name__ == "__main__":
    main()
```

8. Test Cases

ID	Scenario / Input	Expected result
TC1	Opening 100, received 50, distributed 80, damaged 5, recorded 65	Expected 65; discrepancy 0; reconciled
TC2	Expected 32; recorded 28; minimum 50	Shortage 4; replenishment 22
TC3	Expected 40; recorded 45; minimum 30	Surplus 5; replenishment 0
TC4	Recorded stock equals minimum stock	Replenishment 0
TC5	Recorded stock is below minimum stock	Replenishment is minimum minus recorded
TC6	Any quantity is negative	Location rejected/skipped with validation message
TC7	Quantity entered as text	Location rejected/skipped with validation message
TC8	Location count = 0	Program asks for at least one location and stops
TC9	Distributed + damaged exceeds opening + received	Expected stock may be negative; results reveal invalid stock movement totals
TC10	Two or more valid locations	Individual reports and overall totals are displayed

9. Sample Output (Console Screenshot Mock-up)

The transcript below is an illustrative console output, not an image captured from a live terminal. Run the program to reproduce it. If an actual screenshot is required for submission, capture the terminal after running the program.

```
=== Food Supply Chain Stock Reconciliation ===
Number of locations to process: 2

Location 1

Enter details for a food supply location.
Location name: Central Store
Food item: Rice
Opening stock (units): 100
Quantity received (units): 50
Quantity distributed (units): 80
Quantity damaged (units): 5
Recorded remaining stock (units): 65
Minimum desired stock (units): 40

--- Reconciliation Report ---
Location: Central Store
Food item: Rice
Expected remaining stock: 65
Recorded remaining stock: 65
Discrepancy (recorded - expected): 0
Stock status: Reconciled
Replenishment required to reach minimum stock: 0

Location 2

Enter details for a food supply location.
Location name: Community Kitchen
Food item: Wheat
Opening stock (units): 60
Quantity received (units): 20
Quantity distributed (units): 45
Quantity damaged (units): 3
Recorded remaining stock (units): 28
Minimum desired stock (units): 50

--- Reconciliation Report ---
Location: Community Kitchen
Food item: Wheat
Expected remaining stock: 32
Recorded remaining stock: 28
Discrepancy (recorded - expected): -4
Stock status: Shortage of 4 unit(s)
Replenishment required to reach minimum stock: 22

=== Overall Summary ===
Locations processed successfully: 2
Net discrepancy across locations: -4 unit(s)
Total replenishment required: 22 unit(s)
```

10. Conclusion

The Food Supply Chain Stock Reconciliation program provides a structured way to compare expected and recorded food inventory across multiple locations. It calculates stock discrepancies, classifies shortages and surpluses, and determines replenishment quantities needed to meet minimum stock targets. The program demonstrates arithmetic, loops, conditional logic, lists, dictionaries, and input validation in Python. It can be extended to support decimal quantities, expiry-date tracking, batch numbers, automatic transfers, or data storage in a database.

How to run the program

- Save the source code as food_stock_reconciliation.py.
- Open a terminal or Command Prompt in the saved file's folder.
- Run: python food_stock_reconciliation.py
- Enter the number of locations and then the stock details requested for each location.